

11

Cyber Security
Codec Technologies India
Internship 2026

Mobile App Security Testing

Static analysis vs OWASP Mobile Top-10 (2024)

AT A GLANCE

25

Insecure findings

138

Risk score

2 / 18

Critical / High

0

Hardened findings

8 of 10

OWASP cats hit

8 / 8

Tests re-run

self-authored

Target

VERDICT

TESTS 8/8

RE-RUN HERE

SELF-AUTHORED TARGET

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

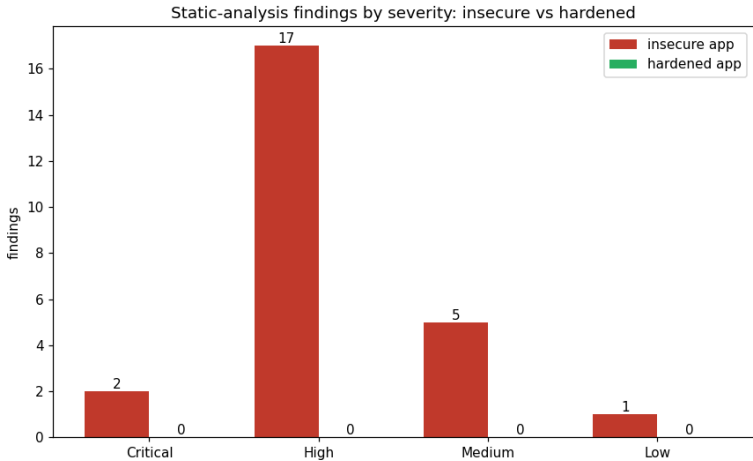
Objective & brief mapping

The brief asked for a mobile app security testing framework. To have something safe to point it at, I also wrote a deliberately-insecure Android sample of my own (and a hardened twin). The framework parses AndroidManifest.xml, scans for hardcoded secrets, weak crypto and insecure WebView/storage, maps findings to OWASP Mobile Top-10 (2024) and MASVS, and produces a MobSF-style report.

Architecture

packager builds an unsigned pseudo-APK (a zip of manifest + sources + res). analyzer orchestrates manifest_analyzer (exported components, backup/debuggable/cleartext flags), secret_scanner (named regexes + Shannon-entropy pass), and code_analyzer (weak crypto, insecure WebView, storage, network-config), then dedups, severity-sorts and renders.

```
insecure_app -> packager -> insecure_demo.apk
analyze_app: manifest + secrets(regex+entropy) + code
-> findings.json + mobsf_style_report.md + severity PNG
```



Findings by severity, insecure app (results/findings_by_severity.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python3 main.py # package -> analyze -> report (~1 s)
python3 unittest-style runner over tests/ # 8 tests
```

The artefacts each figure and table above are drawn from:

- **findings.json** — 25 findings + risk score
- **mobsf_style_report.md** — severity-rated report
- **findings_by_severity.png** — chart
- **insecure_demo.apk** — unsigned pseudo-APK (zip)

Method & implementation

The scanner reads Java/Kotlin source (not compiled DEX), so it can't analyze a closed-source APK — MobSF + apktool/jadx do that decompilation step. Entropy screening is deduped against named patterns and gated with a letters-and-digits heuristic to cut false positives on long identifiers.

Results

From results/findings.json. The same analyzer produces 25 findings on the insecure app and 0 on the hardened one.

Metric	Insecure	Hardened
Findings	25	0
Risk score	138	0
Critical / High	2 / 18	0 / 0
Medium / Low	4 / 1	0 / 0
OWASP cats hit	8 of 10	--

Sample weaknesses -> OWASP Mobile

Weakness	OWASP	Severity
hardcoded AWS/Google/Stripe keys	M1	critical/high
DES+ECB, MD5	M10	high
exported components	M8	high/medium
cleartext + no pinning	M5	high

Verification

Re-ran here, 8/8 (~1 s). Pure static analysis over bundled source, no device needed.

11

Cyber Security
Codec Technologies India
Internship 2026

Mobile App Security Testing

Static analysis vs OWASP Mobile Top-10 (2024)

AT A GLANCE

25
Insecure findings

138
Risk score

2 / 18
Critical / High

0
Hardened findings

8 of 10
OWASP cats hit

8 / 8
Tests re-run

self-authored
Target

VERDICT

TESTS 8/8 RE-RUN HERE

SELF-AUTHORED TARGET

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Test group	Checks	Result
Detection	every planted weakness found	PASS
Tagging	OWASP-Mobile + MASVS on each finding	PASS
Control group	hardened app far fewer findings	PASS
Parity	zip vs directory equal	PASS
Total	8 tests	8/8

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

Static analysis is the deliverable; the dynamic tools it pairs with are documented, not run.

Ran here	Real tool	Note
manifest + secret + code scan	MobSF static	this report is MobSF-style
source-level analysis	apktool / jadx	decompile a real APK first
(static only)	Frida / Burp	runtime hooks, TLS MITM

Ethics, data integrity & declared substitutions

Defensive and educational. I did **not** reverse-engineer anyone else's app — the only target is a sample I built and labelled, and the AWS/Google/Stripe-shaped secrets are fake, non-functional placeholders. **Declared substitution:** dynamic analysis (MobSF DAST, Frida hooks, Burp) is documented in the report but requires a device/emulator and was not run here; this framework is static-only by design.

Learnings

Building the hardened twin as a control group was the smartest decision — "25 vs 0 from the same analyzer" is far more convincing than an absolute count. Combining named-regex secret detection with an entropy pass, then deduping, taught me how real SAST balances recall against the false-positive flood.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- Regex/pattern-based, not full dataflow; reads Java/Kotlin source, not compiled DEX/smali.
- Entropy screening can miss short secrets and false-positive long identifiers (mitigated + deduped).
- The pseudo-APK is a plain zip with no DEX or signature.
- No dynamic analysis; runtime-only issues are out of static scope.
- Insecure app: 25 findings, risk 138; hardened twin: 0 findings, risk 0.
- Both critical findings are hardcoded AWS credentials (fake, non-functional).
- Findings map to OWASP Mobile Top-10 (2024) and MASVS controls.
- The only app analyzed is one I wrote and labelled myself.
- Insecure app: 25 findings, weighted risk 138.
- Severity mix: 2 critical, 18 high, 4 medium, 1 low.
- Hardened twin: 0 findings, risk 0 (same analyzer).
- OWASP categories hit: 8 of 10 (M1 x7 leads).
- secret_scanner combines named regexes with a Shannon-entropy pass.
- packager builds an unsigned pseudo-APK (a zip).
- References: OWASP Mobile Top-10 2024, MASVS, MASTG, MobSF.
- Every planted weakness carries a WEAKNESS source comment.
- Reference: OWASP Mobile Top 10 (2024); OWASP MASVS and MASTG.
- Reference: MobSF; Frida; Android Network Security Config.
- Reference: Android app security best practices.
- findings.json records 25 findings (risk 138) vs hardened 0.
- The only app analyzed is one I wrote and labelled.
- Fake AWS/Google/Stripe-shaped secrets are non-functional.